TP - Système de Gestion Hospitalière avec Entity Framework Core

Contexte professionnel

Vous êtes développeur.NET chez **HealthTech Solutions**, une entreprise qui développe des logiciels pour le secteur médical. Votre mission : concevoir et développer une application de gestion hospitalière complète.

Le chef de projet vous remet le cahier des charges suivant. À vous de traduire ces exigences fonctionnelles en modèle de données EF Core, migrations comprises.

Cahier des charges fonctionnel

Module Patients

- Enregistrer les patients avec leurs informations personnelles (nom, prénom, date de naissance, coordonnées)
- Chaque patient possède un numéro de dossier unique
- Un patient peut avoir plusieurs séjours hospitaliers
- Les informations de contact doivent être complètes (adresse, téléphone, email)

Module Départements

- L'hôpital est organisé en départements (cardiologie, neurologie, pédiatrie, etc.)
- Chaque département a un nom, une localisation et un responsable médical
- Un département peut avoir plusieurs médecins

Module Médecins

- Les médecins ont une spécialité (cardiologue, chirurgien, généraliste...)
- Chaque médecin possède un numéro de licence médicale
- Un médecin appartient à un département et peut suivre plusieurs patients
- Un médecin peut réaliser plusieurs consultations

Module Consultations

- Une consultation associe un patient à un médecin à une date donnée
- La consultation a un statut (planifiée, réalisée, annulée)
- Un patient peut voir plusieurs médecins différents au fil du temps

---

Exigences techniques globales

- **Framework** : ASP.NET Core 8+ (API REST ou MVC)
- **ORM** : Entity Framework Core 8+
- **Base de données** : Au choix (SQL Server, PostgreSQL, SQLite)
- **Architecture** : Clean Architecture recommandée

- **Livrables** : Projet fonctionnel + migrations

Étape 1 : Modélisation initiale

Challenge

Concevez le modèle de données initial permettant de gérer les patients et les départements de l'hôpital.

Contraintes à respecter

- Un patient doit avoir un numéro de dossier unique
- La date de naissance doit être valide (dans le passé)
- L'email doit être unique dans le système
- Les noms et prénoms ne peuvent pas être vides

Travail à realiser

1. Créer les entités Patient et Department
2. Configurer le DbContext
3. Définir les contraintes et validations
4. Créer la première migration et l'appliquer

Questions à répondre dans votre code

- Comment garantissez-vous l'unicité du numéro de dossier ?
- Quelle stratégie utilisez-vous pour les clés primaires ?
- Comment validez-vous la date de naissance ?

Étape 2 : Intégration des médecins

Challenge

L'hôpital embauche des médecins. Chaque médecin doit être affecté à un département.

Exigences fonctionnelles

- Un médecin a une spécialité et un numéro de licence
- Chaque médecin est rattaché à un département
- Si on supprime un département, que deviennent les médecins ?

Challenge additionnel

On souhaitez maintenant que le département connaisse son responsable. Modélisez cette relation.

Travail à réaliser

1. Ajouter l'entité Doctor
2. Définir la relation avec Department
3. Créer une nouvelle migration

Questions de conception

- Quelle relation entre Doctor et Department ?
- Quel DeleteBehavior ? Justifiez votre choix.

Étape 3 : Gestion des consultations

Challenge

Le système doit permettre de planifier et suivre les consultations entre patients et médecins.

Exigences fonctionnelles

- Une consultation associe un patient à un médecin
- Elle possède une date et un statut
- Un patient peut avoir plusieurs consultations dans le temps
- Un médecin peut voir plusieurs patients différents

Défis techniques

- Comment nommer la table de liaison ?
- Quelles colonnes supplémentaires sont nécessaires ?
- Comment éviter qu'un patient ait deux consultations au même moment avec le même médecin ?

Travail à réaliser

1. Créer l'entité de consultation
2. Mapper les relations
3. Créer une migration

Étape 4 : Opérations CRUD et requêtes

Challenge

Maintenant que le modèle est en place, implémentez les fonctionnalités de gestion.

Fonctionnalités attendues

Patients

- Créer un nouveau patient
- Modifier les informations d'un patient
- Supprimer un patient (avec gestion des exceptions)
- Rechercher des patients par nom
- Lister les patients par ordre alphabétique

Consultations

- Planifier une nouvelle consultation

- Modifier le statut d'une consultation
- Annuler une consultation
- Lister les consultations à venir pour un patient
- Lister les consultations du jour pour un médecin

Défis performance

- Comment paginer les résultats ?
- Comment optimiser les requêtes fréquentes ?
- Comment éviter de charger des données inutiles ?

Travail à réaliser

1. Implémenter les services de gestion
2. Écrire les requêtes LINQ
3. Ajouter la pagination

Étape 5 : Chargement des données liées

Challenge

L'équipe médicale a besoin d'afficher des tableaux de bord complets.

Vues à implémenter

1. **Fiche patient** : Afficher le patient + toutes ses consultations avec les noms des médecins
2. **Planning médecin** : Afficher le médecin + son département + toutes ses consultations à venir
3. **Statistiques département** : Pour chaque département, le nombre de médecins et le nombre de consultations

Défis techniques

- Quelle stratégie de chargement utiliser ?
- Comment éviter le problème N+1 ?
- Comment structurer les données pour la vue ?

Questions de performance

- Comparez Eager Loading vs Lazy Loading pour ces cas d'usage
- Quand est-il pertinent d'utiliser des projections ?
- Quel impact sur les performances si on charge toutes les données sans filtre ?

Étape 6 : Modélisation avancée

Challenge

De nouvelles fonctionnalités métier nécessitent une modélisation plus sophistiquée.

Feature 1 : Types complexes

L'adresse d'un patient est utilisée dans plusieurs entités (patient, département pour le contact). Comment factoriser ?

Feature 2 : Héritage

L'hôpital embauche différents types de personnel médical :

- Les médecins (spécialité, licence)
- Les infirmiers (service, grade)
- Le personnel administratif (fonction)

Tous ont des informations communes : nom, prénom, date d'embauche, salaire.

Feature 3 : Hiérarchie de départements

Les départements peuvent être organisés en sous-départements. Par exemple, "Cardiologie" contient "Cardiologie adulte" et "Cardiologie pédiatrique".

Travail à réaliser

1. Identifier les patterns à utiliser
2. Implémenter les changements
3. Créer les migrations correspondantes

Étape 7 : Performance et concurrence

Challenge

L'hôpital est un environnement où la concurrence d'accès aux données est critique.

Exigences

1. **Optimisation** : Les recherches de patients par nom doivent être rapides
2. **Concurrence** : Si deux utilisateurs modifient le même patient simultanément, le système doit détecter le conflit
3. **Index** : Créer les index nécessaires pour les requêtes fréquentes

Queries critiques à optimiser

- Rechercher un patient par son numéro de dossier
- Lister les consultations d'un médecin pour une date donnée
- Compter les patients par département

Travail à réaliser

1. Ajouter les index nécessaires
2. Implémenter le concurrency token
3. Tester les conflits de mise à jour

Étape 8 : Architecture et testabilité

Challenge

L'application va être maintenue et évolue fréquemment. Il faut la rendre testable.

Exigences

1. **Couche d'accès aux données** : Isoler l'accès à EF Core derrière des interfaces
2. **Tests unitaires** : Pouvoir tester la logique métier sans toucher à la base de données
3. **Fabrique de contexte** : Pouvoir créer un DbContext spécifique pour les tests

Patterns attendus

- Repository Pattern
- Unit of Work (si pertinent)
- Dependency Injection

Travail à réaliser

1. Créer les interfaces de repository
2. Implémenter les repositories
3. Écrire des tests unitaires pour la création de patient et la planification de consultation

---

Critères d'évaluation

Compétence	Indicateur
Modélisation	Capacité à traduire les exigences métier en entités et relations
Relations	Maîtrise des relations One-to-Many, Many-to-Many, héritage
Requêtes LINQ	Capacité à écrire des requêtes complexes avec filtres et tris
Performance	Utilisation appropriée de Include, projections, AsNoTracking
Migrations	Compréhension de l'évolution du schéma de données
Architecture	Séparation des préoccupations, testabilité
Autonomie	Capacité à prendre des décisions de conception

Livrables attendus

1. **Projet ASP.NET Core** fonctionnel avec API ou interface MVC
2. **6+ migrations** (une par étape au minimum)
3. **Code source** avec commentaires explicatifs pour les choix techniques
4. **README** avec instructions d'exécution et description de l'architecture

---

Questions de synthèse

À rendre dans un fichier `ANALYSE.md` :

1. Quels sont les avantages et inconvénients de votre modèle ?
2. Quelles optimisations feriez-vous si l'hôpital avait 100 000 patients ?
3. Comment implémenteriez-vous un système de rendez-vous en ligne ?
4. Quel impact sur le modèle si on ajoutait la facturation ?

Deadline : Fin du cours `